Въведение в приложния AI

API-та на водещите езикови модели. Локални модели.

Богомил Стоянов, Михаил Доброславски

Лекция 1

Приложно AI програмиране

© 2026 Богомил Стоянов, Михаил Доброславски · CC BY-NC-ND 4.0

Този курс vs. курс по машинно обучение

Курс по машинно обучение

- Фокус: *създаване* и *трениране* на модели.
- Данни, загуба, оптимизация, оценка.
- Моделът е *обект* на изследване.

Този курс

- Фокус: *изграждане на продукти* около готов модел.
- LLM = **готов API компонент**, не го тренираме.
- Моделът е *инструмент* с договор (contract).

Инженерната гледна точка

LLM е **недетерминиран** компонент, достъпен **по мрежата**, с латентност, ограничения и цена. Външна услуга, не библиотечна функция.

Какво ще построим през курса

- **Стек:** Python и TypeScript/Node.js; реални API-та на OpenAI, Anthropic, Google Gemini; локални модели през Ollama.
- **Резултат (стъпка по стъпка):** structured outputs, RAG системи, tool calling, MCP сървъри, автономни агенти, guardrails.
- **Днес:** мястото на LLM в архитектурата, как изглежда една заявка и кога си струва да изберем локален модел вместо hosted.

Предпоставки

Базови познания по Python/JS; желателно и HTTP/REST/JSON и ООП. **Не** се изискват знания по машинно обучение.

Дневен ред: Лекция 1

Еволюция на приложните интерфейси за LLM

API-та на водещите езикови модели

Локални езикови модели

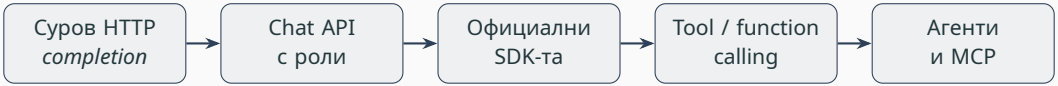
Чести грешки

Обобщение

Дискусия

Еволюция на приложните интерфейси за LLM

Пътят на курса: всяко ниво добавя структура



- Всяко следващо ниво **надгражда** предишното, а не го заменя.
- Курсът се движи отляво надясно: днес сме в началото.

Петте нива в детайл

1. **Суров HTTP / completion API**: един текстов prompt, POST заявка; моделът просто „продължава“ текста статистически, без понятие за роля или разговор (ранните GPT-3 completion endpoint-и).
2. **Chat-базирано API**: разговорът става *списък* от съобщения, всяко с роля (`system`/`user`/`assistant`); OpenAI Chat Completions, 2023. **Повратната точка.**
3. **Официални SDK-та**: типизирани клиенти (`openai`, `anthropic`, `google-genai`), капсулиращи HTTP, auth, retry, streaming.
4. **Function / tool calling**: моделът връща структурирано намерение да извика функция (Тема 9).
5. **Агентни framework-и и протоколи**: LangChain/LangGraph, Vercel AI SDK (Лекция 4), MCP (Лекции 10–11), LangGraph (Лекция 12).

Completion vs. Chat на практика

(1) Completion: суров HTTP, prompt като низ (историческа форма)

```
curl https://api.openai.com/v1/completions \
  -H "Authorization: Bearer $OPENAI_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model": "...", "prompt": "Столицата на България е", "max_tokens": 8}'
# -> моделът просто продължава низа: " София."
```

(2) Chat: списък от съобщения с роли (днешният стандарт)

```
messages = [
    {"role": "system", "content": "Ти си кратък асистент."},
    {"role": "user",   "content": "Коя е столицата на България?"},
]
```

С chat API-тата LLM престава да бъде машина за продължаване на текст и се превръща в компонент с ясно дефиниран интерфейс.

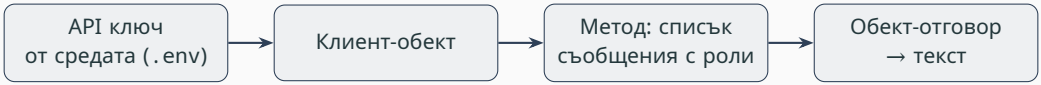
API-та на водещите езикови модели

OpenAI, Anthropic, Gemini

	OpenAI	Anthropic	Google
Python пакет	`openai`	`anthropic`	`google-genai`
Endpoint	Responses (нов) / Chat	Messages API	`generateContent`
System prompt	`instructions` / роля `developer`	**отделен параметър**	в `config`
Таван на изхода	`max_output_tokens` / `max_completion_tokens`	`max_tokens` (**задълж.**)	`max_output_tokens`
Streaming	SSE	SSE	SSE (`alt=sse`)
Tool calling	да	да	да

Ключово послание: общото е по-важно от разликите. Говорим за *абстракции* (роли, параметри за семплиране, streaming, tool calling), не за конкретен доставчик. Това прави възможни *provider-agnostic* архитектури.

Анатомия на заявката: общ шаблон



- Един и същ поток при всички доставчици и на всеки език.
- Разликите са в *детайлите*: къде отива system prompt, кои параметри са задължителни, как се групират настройките.

Модели към 6.10.2026 г. (само в `.env`, не в кода)

Доставчик	Основен	Евтин/бърз	Бележка
Anthropic	`claude-sonnet-5-5`	`claude-haiku-4-5`	спира 15.10.2026
OpenAI	`gpt-6-sol`	`gpt-6-luna`	`gpt-5.1`: deprecated
Google	`gemini-3.8-flash`	`gemini-3.5-flash-lite`	2.5: legacy
Ollama	`llama3.2:1b`	—	слаб на БГ

Променливи: `ANTHROPIC_MODEL`, `OPENAI_MODEL`, `GEMINI_MODEL`, `OLLAMA_MODEL`.
Модел спира ⇒ сменяте един ред в `.env`, не кода.

Минимален пример: OpenAI Responses API (Python)

```python
from openai import OpenAI
client = OpenAI()  # чете OPENAI_API_KEY от средата

# Responses API (препоръчан за нови проекти)
response = client.responses.create(
    model="gpt-6-sol",
    instructions="Ти си кратък и точен асистент.",
    input="Обясни какво е embedding в едно изречение.",
    max_output_tokens=2048,
)
print(response.output_text)
```

GPT-6 са reasoning модели: без `temperature`, дълбочината се управлява с `reasoning_effort`.

Минимален пример: OpenAI Chat Completions (Python)

```python
from openai import OpenAI
client = OpenAI()

# Chat Completions: системната инструкция е съобщение с роля developer
response = client.chat.completions.create(
    model="gpt-6-sol",
    messages=[{"role": "developer", "content": "Ти си кратък и точен асистент."},
              {"role": "user", "content": "Обясни какво е embedding."}],
    max_completion_tokens=2048,    # НЕ max_tokens
)
print(response.choices[0].message.content)
```

По-старият, но все още широко използван API: системната инструкция е *част* от списъка със съобщения.

Минимален пример: Anthropic SDK (Python)

```python
import anthropic

client = anthropic.Anthropic()  # чете ANTHROPIC_API_KEY от средата

message = client.messages.create(
    model="claude-sonnet-5-5",
    max_tokens=4096,              # ЗАДЪЛЖИТЕЛЕН; включва и мисленето
    system="Ти си кратък и точен асистент.",  # САМОСТОЯТЕЛЕН параметър
    messages=[
        {"role": "user", "content": "Обясни какво е embedding в едно изречение."}
    ],
)
if message.stop_reason == "max_tokens":
    print("Отговорът е отрязан.")
# content е списък от блокове; първият може да е "thinking"
print(next(b.text for b in message.content if b.type == "text"))
```

Акцент: `system` е самостоятелен параметър; `max_tokens` е задължителен; текстът се търси **по тип**, не като `content[0]`.

Минимален пример: Gemini SDK (Python)

```python
from google import genai
from google.genai import types

client = genai.Client()  # чете GEMINI_API_KEY / GOOGLE_API_KEY от средата

response = client.models.generate_content(
    model="gemini-3.8-flash",
    contents="Обясни какво е embedding в едно изречение.",
    config=types.GenerateContentConfig(
        system_instruction="Ти си кратък и точен асистент.",
        temperature=0.2,          # настройките се групират в config
    ),
)

print(response.text)
```

Акцент: методът е `generate_content`; настройките отиват в `config`.

Същата заявка, но на TypeScript

```ts
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic(); // ANTHROPIC_API_KEY от env

const message = await client.messages.create({
  model: "claude-sonnet-5-5",
  max_tokens: 4096,
  system: "Ти си кратък и точен асистент.",
  messages: [
    { role: "user", content: "Обясни какво е embedding в едно изречение." },
  ],
});

const block = message.content.find(
  (b): b is Anthropic.TextBlock => b.type === "text",
);
console.log(block?.text);
```

Формата **не зависи от езика**: същите роли и поток, друг синтаксис.

Ключови параметри (преглед; детайли в Тема 2)

- **`temperature`**: степен на случайност/креативност ($0 \approx$ детерминистично, по-високо $\approx$ по-разнообразно); някои по-нови модели не го приемат.
- **`top_p`**: nucleus sampling; алтернативен контрол на разнообразието (най-малкият набор токени с кумулативна вероятност $\geq p$).
- **таван на изхода** (`max_tokens` / `max_completion_tokens` / `max_output_tokens` / `num_predict`): горна граница на дължината на *отговора* (и таван на цената); включва и мисленето.
- **роли** (`system`/`user`/`assistant`): управление на контекста и поведението.

Внимание: `max_tokens` $\neq$ context window

`max_tokens` ограничава само *изхода*; context window-ът е таванът за *вход* + *изход* заедно.

Локални езикови модели

Защо локални модели?

- **Поверителност**: данните не напускат периметъра на организацията.
- **Предвидимост на разходите**: фиксирана hardware инвестиция вместо разход спрямо потреблението.
- **Без мрежова зависимост**: работи офлайн. Латентността зависи от хардуера; на лаптоп обикновено е *по-бавно* от hosted.
- **Контрол върху версията**: доставчикът не може да подмени модела без ваше знание.

Компромис

Най-силните hosted модели обикновено превъзхождат локалните, а собствената инфраструктура има своя цена (GPU/RAM, актуализации, мониторинг).

Пример: Ollama (същата структура на съобщенията)

```
# Инсталация: https://ollama.com
ollama pull llama3.2:1b
ollama run llama3.2:1b "Обясни какво е embedding в едно изречение."
```

```python
import ollama

response = ollama.chat(
    model="llama3.2:1b",
    messages=[{"role": "user", "content": "Кажи ми три факта за Python."}],
)
print(response["message"]["content"])
```

И в облака, и локално се използва същият списък от съобщения с роли. Тази структура е де факто стандарт. `llama3.2:1b` е малък и слаб на български; при възможност сменете `OLLAMA_MODEL` с по-голям многоезичен модел.

Hosted vs. локални модели

	Hosted API	Локален (Ollama и др.)
Качество на топ модел	Най-високо	По-ниско (но расте бързо)
Цена	Спрямо потреблението	Фиксирана hardware инвестиция
Поверителност	Зависи от доставчика	Пълен контрол
Латентност	Мрежов round-trip + опашка	Зависи от хардуера
Поддръжка	Грижа на доставчика	Собствена инфраструктура

Хибриден подход в практиката

Routing: простите задачи и тези с чувствителни данни отиват към локален модел, а сложните към hosted. Изборът е скрит зад единен интерфейс; извикващият код не се променя.

Чести грешки

Чести грешки: конфигурация на заявката

- **API ключ в git / в кода**: сериозен пробив; ключът се пази в `.env` (в `.gitignore`) или secret manager.
- **`max_tokens` ≠ context window**: първото ограничава само изхода.
- **Забравен `max_tokens` при Anthropic**: суров HTTP → 400; SDK-то дава грешка още в клиента.
- **`content[0].text`**: първият блок може да е `thinking`; търсете текстовия блок по тип и проверявайте `stop_reason`.
- **System prompt на грешно място**: OpenAI: роля `developer` или `instructions`; Anthropic: самостоятелен параметър; Gemini: в `config`.
- **Модел, вписан в кода**: моделите се спират; дръжте ги в `.env`.

Чести грешки: очаквания и архитектура

- **Очакване на пълен детерминизъм**: дори при `temperature=0` изходът може да варира.
- **„Локален = безплатен и по-добър“**: има реален хардуерен/оперативен разход и обикновено по-ниско качество на топ ниво.
- **Vendor lock-in по невнимание**: тънък адаптерен слой отрано ограничава щетата.
- **Игнориране на rate limits / retry**: грешките 429 и временните откази трябва да се обработват с повторни опити и backoff.

Обобщение

Какво научихме днес

- В този курс LLM е **външен API компонент**, не обект на трениране.
- API-тата еволюираха: суров HTTP → chat → SDK-та → tool calling → агенти. Всяко ниво добавя структура.
- OpenAI, Anthropic и Gemini споделят обща форма (роли, параметри за семплиране, streaming, tool calling); разликите са в детайлите.
- Локалните модели (Ollama) са реална алтернатива при поверителност, офлайн работа или контрол на разходите; на практика се ползва хибриден routing.

Следваща лекция

Системен дизайн на AI приложение: параметри за семплиране и роли в дълбочина, context management и промпт инженерство на ниво код.

Връзка с изпитния конспект

- **Въпрос 1**: „Еволюция и видове приложни програмни интерфейси за езикови модели (API).“ → покрит от секции 1 и 2.
- **Въпрос 2**: „Параметри на LLM при извикване: Temperature, Top-P, Max Tokens.“ → въведен тук; пълно разглеждане в Тема 2.
- **Въпрос 3**: „Роли и управление на контекста: System, User, Assistant.“ → въведен тук; задълбочаване в Тема 2.

Дискусия

Дискусия: правила

В края на всяка лекция отделяме време за обща дискусия (≈10–15 мин).

- Критикуваме *идеи и аргументи*, не хора; уважаваме различните мнения.
- Кратко и аргументирано: подкрепи тезата с пример, опит или контрапример.
- Няма „глупави“ въпроси и грешни отговори. Целта е да мислим на глас; „не знам“ е напълно приемливо.
- Слушаме активно: един говори в даден момент; надграждаме казаното от другите.
- Никой не е длъжен да участва, но всеки има думата.

Теми

1. **Локален vs. hosted в реален проект.** Да вземем болница, стартъп и банка. Какво натежава при всеки от тях: цена, поверителност, качество или поддръжка?
2. **Недетерминизъм.** Как фактът, че LLM компонентът е недетерминиран, променя начина, по който тестваме и гарантираме качество спрямо класическа детерминистична функция?
3. **Provider-agnostic абстракция.** Ранен адаптерен слой: разумна застраховка срещу vendor lock-in или преждевременна оптимизация? Какво накланя везните?

Библиография

- Alammar & Grootendorst (2024). *Hands-On Large Language Models*.
- OpenAI API Developer Reference: `https://developers.openai.com/api/docs/`
- Anthropic Claude Developer Documentation: `https://docs.claude.com/`
- Gemini API Documentation: `https://ai.google.dev/gemini-api/docs`
- Ollama Documentation: `https://github.com/ollama/ollama`